Thermal vision AI reference application

Rob Liston, Fawad Ahmad, Zafar Ali, Jash Garish

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Contents

Introduction

This document defines the architecture and microarchitecture for an FPGA reference application targeting the Gemini device. The application utilizes the MLX90640 IR thermal sensor array from Melexis, which produces a 32x24 array of high precision thermal pixels at 32Hz. This thermal sensor interfaces directly to the FPGA fabric using an I2C interface over GPIO. A 14-layer CNN with 200K parameters is implemented in the FPGA fabric using CLB, BRAM and DSP resources. The output of the CNN is a 32Hz stream of predictions which can be accessed by the SoC CPU using a memory mapped interface. The CNN can be trained to perform a variety of classification tasks. All of the weights and data buffering are stored in BRAM in the FPGA fabric. In the reference application, the predictions and raw sensor values are encapsulated in UDP IP packets and sent out the SoC ethernet interface. The system hardware diagram is shown in Figure 1.

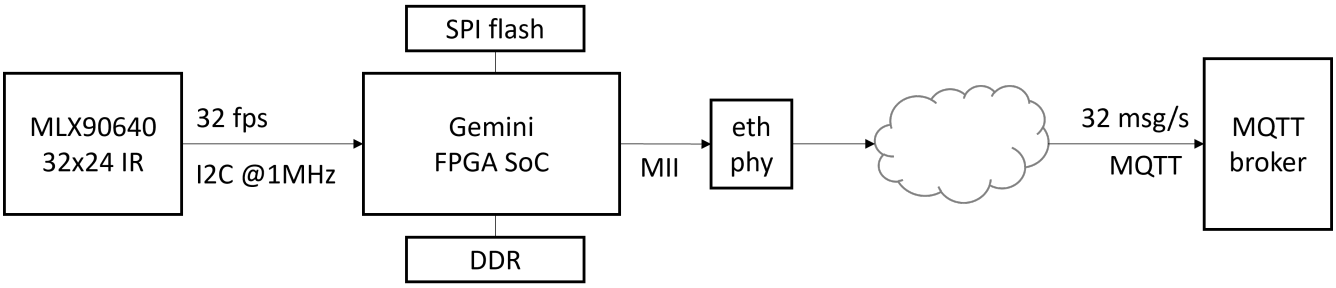

Figure 1. System Diagram

1. FPGA gateware

The FPGA fabric design consists of three major modules as shown in Figure 2. The ibuf and obuf module each have one instance and the conv2d module contains one instance per CNN layer. The evaluation of the CNN is fully pipelined and does not contain any feedback loops or stall mechanisms. In this design, all tensors are composed of 18 bit signed integers and are serialized in 'C' order, for example in BRAM memory or over AXI-Stream interfaces.

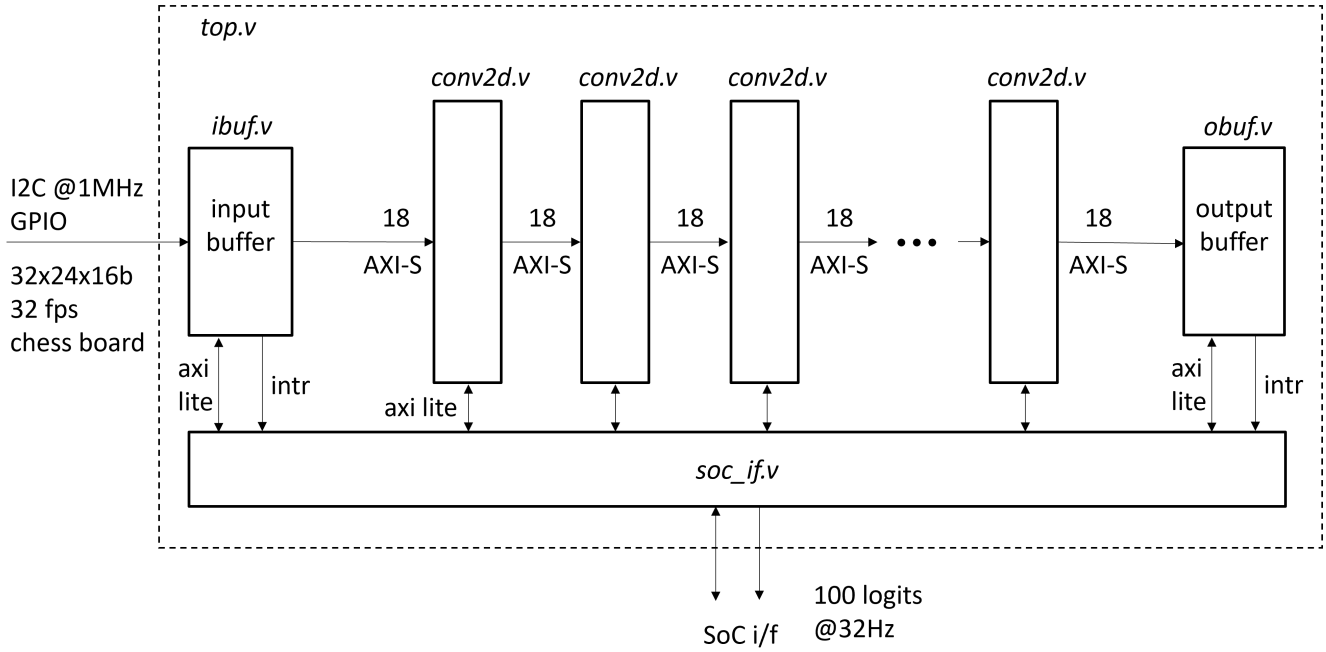

Figure 2. FPGA Top Level

Clocks The andromeda.v design contains two clock domains. The `sys_clk` domain is used to clock the AXI slave interfaces, ibuf.v, obuf.v and soc_if.v and runs at 100MHz. The `dsp_clk` domain runs at 400MHz and is used to clock the AXI-S stream interfaces and the conv2d.v modules.

1.1 Input buffer

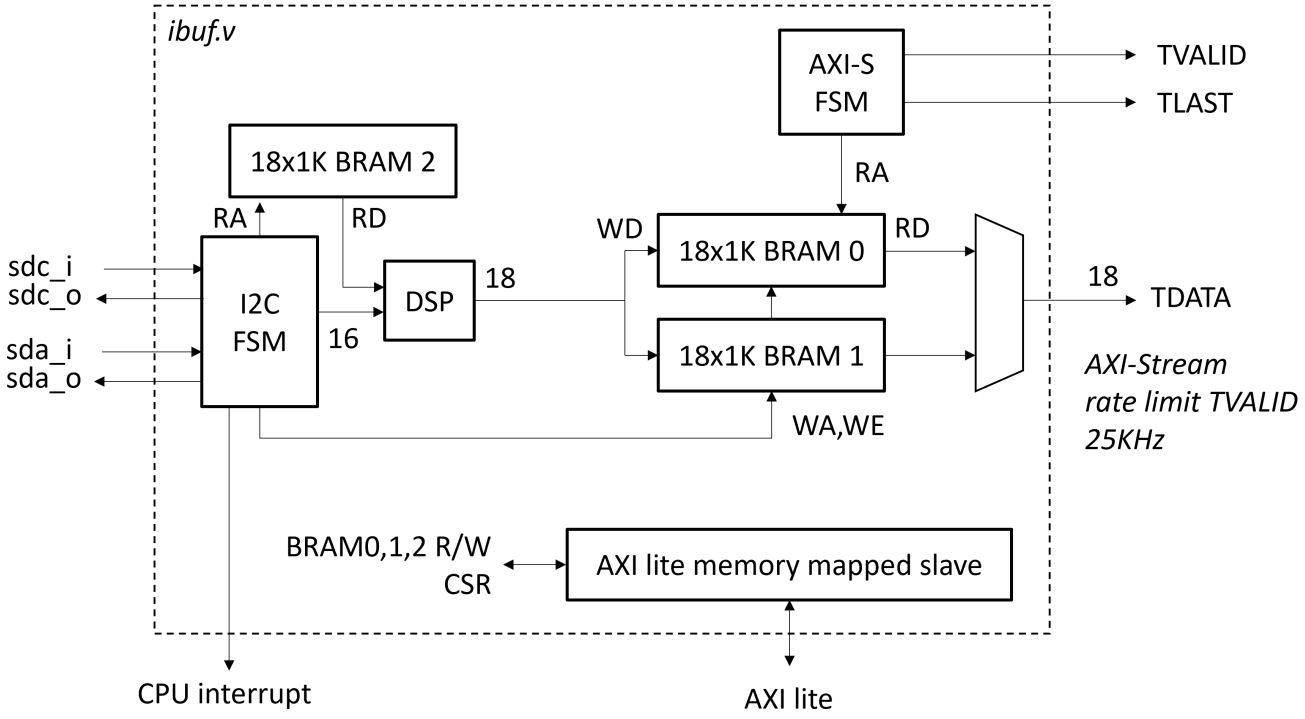

Figure 3. Input buffer

The block diagram of the input buffer module is shown in Figure 3. BRAM 0 is used to store complete even frames and BRAM 1 stores complete odd frames, where a frame is an array of pixels with shape [24,32,1]. All values are 18 bit signed integers. These BRAMs contain 2 write ports and 2 read ports. One write port is used by the I2C FSM to store and deinterlace incoming pixels from the sensor. One read port is used by the AXI-S FSM to read pixels in raster order and send them out the AXI-S interface. The second read and write ports are used to provide a memory mapped interface to the pixel values. The input buffer module contains the following memory map which is accessible through the AXI memory mapped slave interface.

Table 1. Input buffer memory map

Address	Bits	RW	Reset	Description
0x0	0	W	1	Bit bang SDA out (open drain)
0x0	1	R	-	Bit bang SDA in
0x0	2	W	1	Bit bang SCL out (open drain)
0x0	3	R	-	Bit bang SCL in
0x4	0	R/W	0	Run/Stop
0x1000	17:0	R/W		BRAM 2 EEPROM calibration coefficients for pixel compensation
0x2000	17:0	R/W		BRAM 0 SoC CPU access
0x3000	17:0	R/W		BRAM 1 SoC CPU access

I2C FSM The ibuf.v module contains an I2C master FSM which periodically reads IR pixel values from the sensor, applies per-pixel compensation factors, and stores the pixels in an on-chip 32x24 BRAM frame buffer. Refer to the MLX90640 data sheet. for details on sensor initialization, reading pixel values, and applying compensation factors. The bit bang interface is used by the SoC application CPU to initialize the MLX90640 sensor. The sensor should be initialized with factory default settings except for the IR refresh rate which should be set to 64Hz. Note that 64Hz is the interlaced rate for one subframe, the actual frame rate is 32Hz. It is desirable to make the refresh rate configurable in order to trade off thermal noise against refresh rate. At initialization, the calibration parameters must be read from the MLX90640 EEPROM using I2C reads, mathematically combined into 768 per-pixel coefficients, and stored in BRAM 2. Initially these coefficients can be set to unity to bypass

calibration compensation. The compensation procedure is described in section 11.2.2 of the MLX90640 specification. Pixel compensation will require the use of a DSP multiplier and fixed point arithmetic. For simplicity, the input buffer can be double buffered so that one BRAM is receiving from the I2C FSM while the other is sending to the AXI-S FSM. Note that a 768-pixel frame fits in one 18x1K BRAM. The overall pseudo code for the I2C FSM follows.

1. Poll MLX90640 until new data available
2. Read subframe 0 pixels from sensor RAM starting at 0x400
3. Multiply each pixel by its individual compensation coefficient (768 fixed point coefficients stored in BRAM 2)
4. Store compensated pixels in BRAM 0 (even frames)
5. Repeat 1-4 for subframe 1 (chess pattern interleave)
6. Assert interrupt to SoC CPU, trigger AXI-S FSM
7. Repeat 1-6 for BRAM 1 (odd frames)

Note that if the input pixels arrive in raster order there is no need for a frame buffer. However, the MLX90640 sensor captures pixels in an interlaced chess board format. The frame buffer is designed to deinterlace the two chess boards into a single complete frame and then sequentially send each pixel over the output AXI-S interface in raster order. The frame buffer uses double buffered BRAM so the I2C side and AXI-S side can operate simultaneously. Additionally, the ibuf.v module generates an interrupt to the SoC CPU when a complete frame arrives (at 32Hz), so that the CPU can read the 768 raw pixel values in real time.

AXI-S FSM The AXI-S FSM operates concurrently with the I2C FSM and is responsible for emitting frames over the AXI stream interface. The overall pseudo code for the AXI-S FSM follows.

1. Receive frame 0 trigger signal from I2C FSM `foo_bar`
2. Sequentially read pixels from BRAM 0 in raster order and output to AXI-S
3. Rate limit AXI-S so that TVALID toggles at 25KHz (768 pixels * 32Hz)
4. Assert TLAST
5. Repeat 1-4 for frame 1 / BRAM 1

Conceptually, data on the AXI-S interface is a tensor sent sequentially in 'C' order and framed using TLAST. The shape of the IR sensor data is [24,32,1]. The pixels should be in raster order with 0,0 in the upper left corner of the frame.

1.2 Conv2d layer

The conv2d module is a parameterized Verilog module which is responsible for evaluating one layer in a deep model. The module parameters are defined in Table 2.

Table 2. Conv2d module parameters

Parameter	Limitations
ichan	tbd
iwidth	tbd
ochan	tbd
kheight	tbd
kwidth	tbd
stride	tbd

The conv2d block implements the functional equivalent of

```
tf.keras.layers.Conv2D(filters=ochan, kernel_size=[kheight,kwidth],
strides=[stride,stride], padding='valid', activation='relu', use_bias=True,
data_format='channels_last').
```

The conv2d module is responsible for evaluating one layer in a deep CNN model as depicted logically in Figure 4. On the input side, incoming features are stored in a circular row buffer which contains kheight rows corresponding to the convolutional kernel size. The incoming features arrive in 'C' order with overall shape $[iheight, iwidth, ichan]$, ending with TLAST. Each time a complete patch is formed in the row buffer, the output side iterates the two read port addresses to feed the DSP with the dot product input vectors, each with shape [kheight,kwidth,ichan], to calculate a single output feature. This process is repeated for ochan weight patches. The output of the block is a sequence of output features, computed in 'C' order with overall

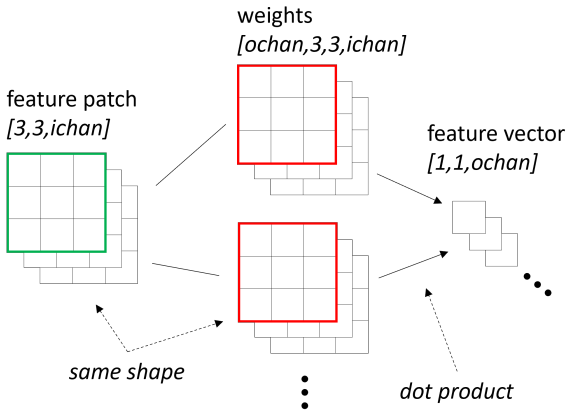

Figure 4. System Diagram

shape $[ceiling((iheight-kheight+1)/stride), ceiling((iwidth-kwidth+1)/stride), ochan]$, ending with TLAST. A RELU nonlinearity is applied and the result is sent to the output AXI-S interface. A variable number of conv2d layers can be connected in a pipeline using AXI-S interfaces. The AXI-S interface is simplified because there are no stalls in the pipeline, so TREADY is not required. Each layer in the model is parameterized by $iwidth, ichan, stride, ochan, kheight, kwidth$ where the input shape is $[iheight, iwidth, ichan]$ and the output shape is $[ceiling((iheight-kheight+1)/stride), ceiling((iwidth-kwidth+1)/stride), ochan]$. Depending on these parameters, the conv2d instantiation may contain a variable number of BRAM instances. Each BRAM is 18x1K and the total storage requirement is $18*(ichan*iwidth*kheight+ichan*kheight*kwidth*ochan)$. In this version of the design there is always a single DSP multiply accumulate unit per layer, which provides a per layer MAC budget of 400M MAC/s. Refer to spreadsheet [1] for details.

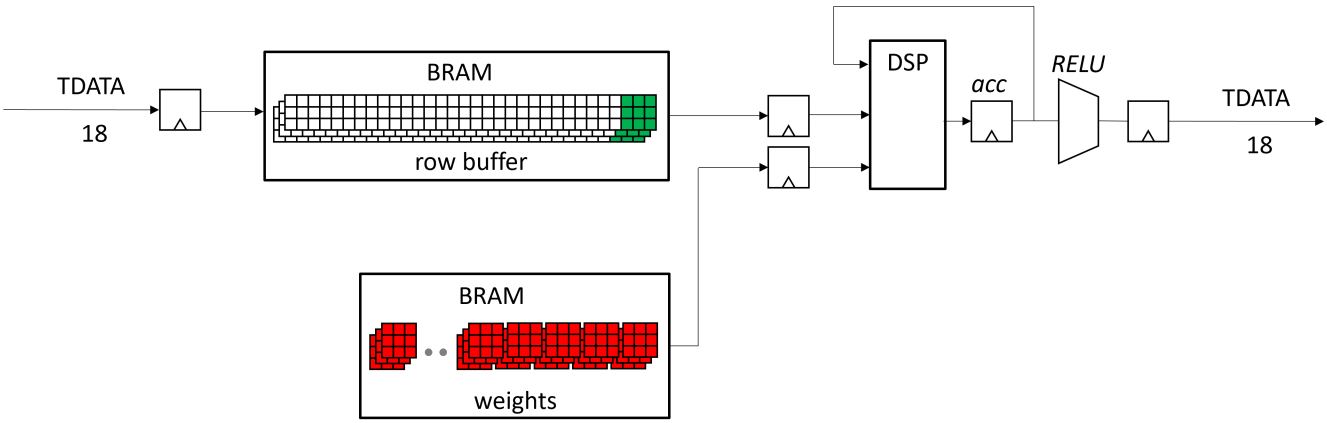

Figure 5. System Diagram

The conv2d block diagram is shown in Figure 5. The incoming data is stored in a row buffer using BRAM 0...n, with shape [kheight,iwidth,ichan]. The weights for the layer are also stored in BRAM 0...n with shape [ochan,kheight,kwidth,ichan]. The total depth for BRAM 0:n is kheight*iwidth*ichan + ochan*kheight*kwidth*ichan. As complete patches are received into the row buffer with shape [kheight,kwidth,ichan], the BRAM iterator iterates the RA and RB read port addresses to read the data and weights for the dot product computation by the DSP MAC. There are ochan weights, each with shape [kheight,kwidth,ichan]. The dot product for each ochan is computed and emitted serially so that the output shape is [ceiling((iheight-kheight+1)/stride), ceiling((iwidth-kwidth+1)/stride),ochan]. The reduction in height and width is due to the use of padding='valid'. Note that the conv2d module can process frames with arbitrary iheight. After the dot product is computed for each ochan, a RELU activation function is applied before sending TDATA out.

Figure 6 shows how the row buffer and weights are organized in the BRAM for kheight=3, kwidth=3. Incoming frames from the AXI-S input interface are circularly stored into the row buffer in 'C' order with shape [3,iwidth,ichan]. When a complete [3,3,ichan] patch is available, the BRAM iterator uses the two BRAM read ports RA and RB to read the row buffer

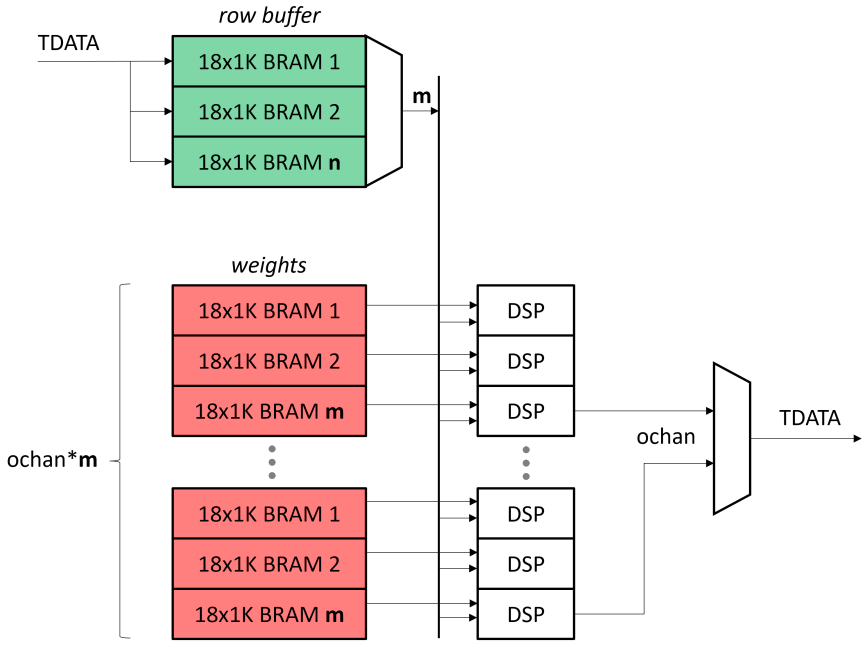

Figure 6. System Diagram

and weights in order to successively compute ochan dot products. A total of ochan dot products are generated per input patch to form an output feature vector. For each dot product, the same row buffer patch is multiplied by ochan different weight patches. All of the dot products must be finished before the next row buffer patch arrives in order to eliminate the need for stalls.

1.3 Output buffer

The output buffer contains a double buffer BRAM which stores the final CNN prediction output shape, generates an interrupt to the SoC CPU, and provides memory mapped AXI slave access to the prediction buffer.

1.4 SoC interface

The soc_if.v module is responsible for AXI slave decoding and interrupt control.

2. Application software

TBD: update this section to use MQTT transport

Embedded software The application software is interrupt driven. One interrupt is used to signal the arrival of a new frame in the input buffer. The application can read the pixel values using the AXI memory mapped slave interface, copy to a UDP packet buffer, and transmit out the ethernet SoC interface. Similarly, another interrupt indicates that a new prediction is available to be read from the output buffer. A complete frame of raw pixels requires 768*2 bytes which will require two UDP packets. The prediction frame requires 100*2 bytes assuming a final CNN shape of [1,1,100]. The UDP packets are sent to a configurable IP address and UDP destination port. The application software also performs MLX90640 sensor initialization, EEPROM calibration data processing, and programming compensation coefficients to BRAM. The application software also programs the model weights into the conv2d BRAM at initialization.

Cloud software The server software is a python program which listens to a UDP port, receives packets with raw pixels and predictions, and renders a live visualization.

3. Deep model

For this application, the following deep model will be implemented.

Model training and weight extraction The conv2d layer can be used to build general purpose deep models which can be trained using standard frameworks such as tensorflow, keras, or pytorch. The overall procedure for developing and deploying a deep model is as follows. 1. Create tensorflow model with the same structure and conv2d layer parameters as the Verilog

```
tensorflow 2.7.0
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 24, 32, 1)]       0
 conv2d (Conv2D)             (None, 22, 30, 32)        288
 conv2d_1 (Conv2D)           (None, 20, 28, 32)        9216
 conv2d_2 (Conv2D)           (None, 18, 26, 32)        9216
 conv2d_3 (Conv2D)           (None, 16, 24, 32)        9216
 conv2d_4 (Conv2D)           (None, 14, 22, 32)        9216
 conv2d_5 (Conv2D)           (None, 12, 20, 32)        9216
 conv2d_6 (Conv2D)           (None, 10, 18, 32)        9216
 conv2d_7 (Conv2D)           (None, 8, 16, 32)         9216
 conv2d_8 (Conv2D)           (None, 6, 14, 33)         9216
 conv2d_9 (Conv2D)           (None, 4, 12, 32)         9216
 conv2d_10 (Conv2D)          (None, 2, 10, 32)         9216
 conv2d_11 (Conv2D)          (None, 1, 1, 100)         64000
 conv2d_12 (Conv2D)          (None, 1, 1, 100)         10000
 conv2d_13 (Conv2D)          (None, 1, 1, 100)         10000
=================================================================
Total params: 176,448
Trainable params: 176,448
Non-trainable params: 0
_________________________________________________________________
```

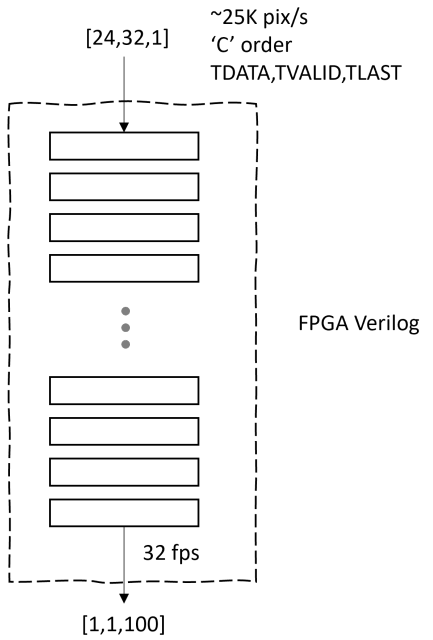

Figure 7. System Diagram

code 2. Train the model using tensorflow 3. Extract the weights from the trained model 4. Convert floating point weights to fixed point 5. Program weights into conv2d Verilog BRAM for each layer 6. The output of the last conv2d layer represents the predicted logits which must be further processed using e.g. softmax in order to generate a probability distribution.

Acknowledgments

So long and thanks for all the fish [1, 2].

References

[1] Rob Liston. conv2d.v resource and performance spreadsheet. https://rapidsilicon2.sharepoint.com/:x:/s/ApplicationEngineering/EUEVBp6TAQ9Mj7beGsrXD1kB0c3MX7wFG66fw589haIPXQ?e=T7h3YU, 2022. [Sharepoint, need to move to git].

[2] MLX90640 specification. https://www.melexis.com/en/documents/documentation/datasheets/datasheet-mlx90640.